

Campus Visitor Pass Architecture Decision

Bay Atlantic University
CMPS 570 Software Architecture
Week 6 Security and Testability
October 2 2026

Prepared for team review by Muhammed Enver Altadfi. Add the confirmed teammates and their actual contributions in README.md before submission.

Choice and scope

The recommended full system is a layered monolith: one deployable server contains HTTP and authentication middleware, a Pass module, an Audit module, and repository adapters. A browser presents the Visitor, Host and Guard views. A private database stores requests and decisions. REST endpoints may expose the application, but REST describes the interface rather than a separate deployment style.

The classroom proof implements the same separation inside one HTML page using JavaScript. It uses memory and simulated roles as the brief permits. It demonstrates service-level permission checks and dependency injection. Anyone can select the Host role or alter browser memory, so the prototype is not a production security boundary. The proposed server, staff authentication, HTTPS and database encryption are design requirements, not implemented features of this proof.

Quality attributes and rationale

Security. A single server entry point authenticates staff, validates input and limits requests. The Pass module checks role and host assignment on each decision. The Guard receives only validity and visit date. Audit records omit names and pass codes. The database accepts only the application service account.

Testability. The Pass module receives its repository, clock, actor provider, code generator and audit sink from outside. Tests control state and time without real visitors or a database. Clear return values and sanitized events make failures observable.

Maintainability. Small internal modules keep visit rules separate from screens and storage. A future SQL repository can replace memory without rewriting approval rules. The trade-off is one deployment and failure domain, and scaling the application as a whole.

Alternatives and trade offs

Microservices rejected. Independent scaling could help a larger campus deployment, but this short workflow would gain network failure paths, distributed authorization, more deployments and harder integration tests. The one-day team deadline favors fewer moving parts.

Event-driven architecture rejected. Approval events could support notifications and other integrations. For the core gate decision, asynchronous delivery introduces eventual consistency, duplicate-event handling and uncertainty about when a pass becomes valid. The proof uses an immediate state transition. Notifications could later be an auxiliary event consumer.

Components and trust boundaries

CAMPUS VISITOR PASS PROPOSED FULL SYSTEM

Layered monolith. Modules inside the server box deploy together.

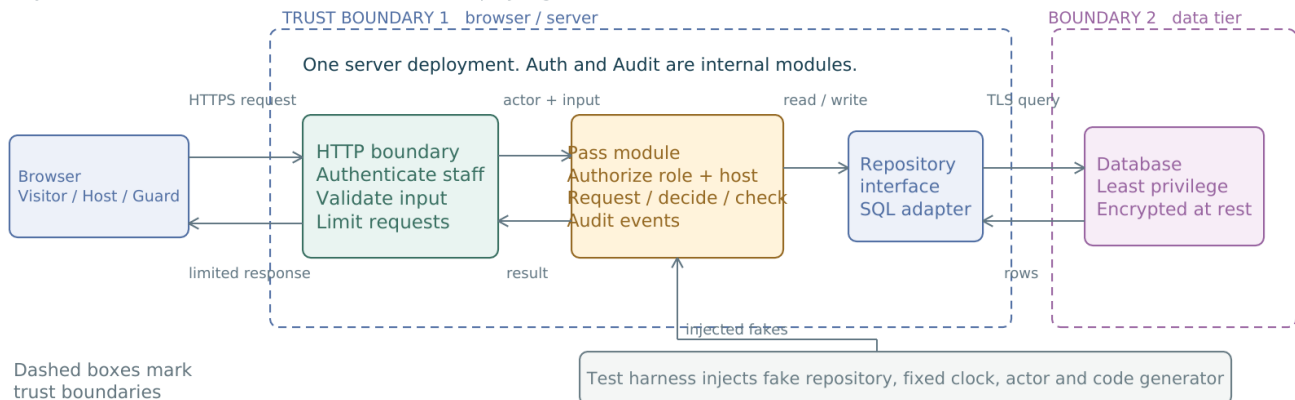


Figure 1 shows the proposed full system. The HTTP boundary authenticates staff before creating a trusted actor context. Public visitor requests still receive validation and rate limits. The Pass module calls repository methods, and the SQL adapter uses parameterized queries. Audit is an internal module called by the Pass module. Arrow labels describe messages crossing interfaces.

The first dashed boundary separates untrusted browser input from server logic. The second separates the application from the restricted data tier. Internal modules have separate responsibilities but share a server process. Tests inject dependencies into the Pass module through protected development interfaces.

Workflow and data

A Request contains an ID, visitor name, visit date, assigned host, status and optional pass code. Its state moves from pending to approved or denied once. Approval generates a unique code. A Guard check succeeds only for an approved pass on its visit date. The prototype uses a 12-character code from a 32-symbol alphabet with browser cryptographic randomness. The full system should store a code hash, expire codes after the visit day, throttle checks and apply a documented retention policy.

Security scenarios and tactics

S1 Unauthorized approval. Source: a Visitor, Guard or different Host. Stimulus: an approval request targeting another host's pending visit. Environment: normal operation. Artifact: Pass module and request record. Response: reject before mutation and write a sanitized audit event. Measure: 100% of unauthorized test attempts are rejected and create no code, with the audit event emitted during the call. Tactics: authenticate actors, authorize actors, limit access and audit. The full server should return HTTP 403. The proof throws an access-denied error.

S2 Personal data exposure. Source: an authenticated Guard. Stimulus: verifies a pass and attempts to obtain its visitor record. Environment: normal operation. Artifact: verification response and logs. Response: return only validity and visit date, deny access to host lists, and exclude names and codes from logs. Measure: zero visitor names or host details in Guard verification responses and audit events in tests. Tactics: authorize actors, limit exposure, validate input and audit. HTTPS and encryption at rest provide additional protection in the full system.

Testability scenario

T1 Repeatable approval and expiry tests. Source: automated tester. Stimulus: a change to approval or date rules. Environment: development and CI. Artifact: Pass module. Response: execute assertions with a fresh memory repository, fixed date, fake actor and controlled code generator, then report failures. Measure: all service tests finish in under five seconds without network access or real visitors. Tactics: abstract data sources, localize state, executable assertions and limit nondeterminism. Observability uses explicit results and an injected audit sink. The supplied suite has 18 passing service tests; the five-second budget is a design target, not a production performance claim.

ADR 001 Layered monolith

Context. A small team has one day to prove a request, decision and gate-check workflow. Visitor personal data needs controlled access. There is no evidence that independent scaling is necessary.

Decision. Use one server deployment with HTTP/authentication, application, audit and repository layers. Keep dependencies directed toward application interfaces. Store the workflow in one database and make approval atomic in the full system.

Consequences. Authorization and transactions stay close to the visit rules. Unit tests isolate the Pass module. Deployment and diagnosis require fewer services. A server outage affects the whole workflow, and later independent scaling would require extracting modules. This cost is accepted for the current scope.

ADR 002 Inject storage and security context

Context. Tests must run without real visitors. The proof must run as one page without a server, but its design must explain how trusted authentication would work in a full system.

Decision. Inject a repository, actor provider, date provider, code generator and audit sink. Use a fresh memory repository in the proof and tests. Enforce permissions in application methods, including the assigned-host rule. In the full system, authenticated server middleware supplies actor identity. Use protected test environments for substitutions.

Consequences. Tests can select users, dates, stored records and random-code collisions. The adapter adds wiring code and contract tests will be needed for a SQL implementation. Simulated roles and memory cannot prevent browser tampering. Production must remove the role switcher, protect test interfaces, authenticate staff and enforce checks on the server.

Proof and presentation

Open index.html. Create a fictional visit for today. Try approval as Visitor and observe the denial. Switch to the assigned Host, approve and copy the code. Switch to Guard and check a wrong code and the approved code. Also show rejected input and a denied request. Use about three minutes to defend the architecture and ADR 001, followed by a two-minute demonstration.

Source: Dr. Tsige Tessema, Week 6 Security and Testability lecture, especially slides 8-10 and 13-20. The lecture cites Bass, Clements and Kazman, Software Architecture in Practice, fourth edition, chapters 11-12. The team-project brief supplies the workflow and submission requirements.